The effects of AI recommendation on epistemic diversity

Drafting-comments edition. Phase 1 (the unaided two-armed bandit of §1–2) is implemented in `papers/recommenders/model/`: `BernoulliBandit`, `BetaBernoulliAgent` with independent Beta priors per arm, a `NullRecommender` stub, three candidate definitions of D_i in `discovery.py`, and an orchestrator `run_simulation` (reproducible via `SeedSequence`-spawned per-agent RNGs). Notebook `experiments/1. Phase 1 - Unaided Bandit.ipynb` reproduces §2. Phase 2 (the recommender, §3–5) is wired but not yet behaviourally implemented.

I describe a simple model (sections 1–3), make some guesses about how the model may behave (section 4), and discuss some potentially philosophically interesting consequences of these conjectures (sections 5–6).

1 Standard two-armed bandit

Start with a standard two-armed bandit model. There are two arms, A and B. Arm A has mean payoff p_A. Arm B has mean payoff p_B. Assume $p_A > p_B$, so A is the better arm.

Implemented as `BernoulliBandit(p_a, p_b)` with strict $p_A > p_B$ validation. Pending modelling choice: the paper does not specify the payoff distribution; the Beta-prior choice below strongly implies *Bernoulli* rewards via Beta-Bernoulli conjugacy, and that is what the code assumes. If we want continuous payoffs in $[0,1]$ instead, the conjugate update changes and `BernoulliBandit.pull` needs revisiting.

There are n agents. Assume for now that agents do not communicate or observe others' payoffs. Each agent has a Beta prior over p_A and p_B. Agents condition on their observed payoffs.

Implemented as `BetaBernoulliAgent` with two *independent* Beta posteriors, one per arm (row i of `alpha_beta` is $[\alpha_i, \beta_i]$). Default prior is uniform $\text{Beta}(1,1)$ per arm. Pending: do we want a non-uniform shared prior across the community? Currently all agents start from the same prior.

Every agent is a myopic Bayesian. At every stage, they choose whichever arm has the higher posterior expected payoff.

Implemented in `BetaBernoulliAgent.choose_arm`: pick the arm with higher posterior mean $\alpha_i/(\alpha_i+\beta_i)$; ties broken uniformly at random with a per-agent RNG.

Let D_i be the event that agent i discovers which is the better arm (this needs to be defined rigorously eventually). The community succeeds (D) if at least one agent succeeds, so

$$D = D_1 \cup \cdots \cup D_n.$$

Decision still open. Three operational definitions are implemented in `model/discovery.py` and can be swapped at evaluation time: (i) *last-k*: agent has chosen A on the last k steps; (ii) *posterior gap*: $E[p_A \mid h] - E[p_B \mid h] \geq \delta$; (iii) *posterior probability*: $P(p_A > p_B \mid h) \geq 1 - \alpha$ via Monte Carlo. The phase-1 notebook estimates $\hat{q}$ under all three so we can decide which one matches the postulate $q \approx 0.05$ below, and the rigorous definition the paper needs.

2 Unaided collective success

I use P for the "objective" probabilities determined by the model specifications. Suppose that for all i

$$P(D_i) = q = 0.05.$$

I don't know how plausible this is, but the point is we're not assuming that any agent is very likely to succeed.

$q = 0.05$ is a *postulate*, not derived from p_A, p_B or T. The phase-1 notebook reports $\hat{q}$ as a function of $(p_A, p_B, T, \text{prior}, \text{definition})$ so we can either back out parameters that yield $q \approx 0.05$, or argue from the model that some other value is the right anchor.

Because the agents are independent, the probability that the community succeeds is much higher than the probability of individual success. With $n = 100$ agents,

$$P(D) = 1 - (1-q)^n = 1 - 0.95^{100} \approx 0.994.$$

Empirically verified in the phase-1 notebook: $\widehat{P(D)}$ (fraction of trials with at least one D_i true) and the closed-form $1 - (1-\hat{q})^n$ are reported side by side and should agree, since the unaided agents are genuinely independent (no shared state).

3 AI recommender

Now introduce an AI recommender. At each stage t, agent i gives the recommender her private history h_i^t (which arms she has pulled and which payoffs she has received). By conditioning on that history, the agent also has posterior π_i^t. The recommender ρ takes these two things as inputs and returns a recommendation

$$R_i^t := \rho(h_i^t, \pi_i^t, Z) \in \{A, B\}.$$

Interface implemented in `model/recommender.py`: abstract `Recommender.recommend(agent_id,`

`history, posterior_alpha_beta, rng)` returns an optional `Recommendation(arm, likelihood)`. Phase 1 ships `NullRecommender`, which always returns `None` — i.e. the unaided regime. Phase 2 needs a concrete subclass.

The variable Z represents the features of the recommender that are common among all agents, e.g. the algorithm that ρ uses to make recommendations. Suppose (here is a big simplification) $Z \in \{G, F\}$, that is, Z can be in two states (a good state or a failure state). Suppose

$$P(Z = G) = 0.99, \qquad P(Z = F) = 0.01.$$

Pending Phase 2 design: Z is drawn once per simulation (per community) inside the concrete recommender's constructor, and is read by `recommend(...)` when producing each R_i^t. The shared `recommender_rng` already spawned in `run_simulation` is the natural place to draw it. We also need to record Z on the `SimulationResult` so per-trial conditioning $(P(D) \mid Z)$ can be reported empirically.

Interpretive question about Z, still unresolved. The paper writes $P(Z = G) = 0.99$, which reads like an exogenous draw, but the prose says Z "represents the features of the recommender", i.e. the algorithm's design — something fixed, not random. Which is meant? Several readings are compatible with the maths:

1. *Per-community draw of recommender identity*: each community gets a fresh recommender whose hidden type is sampled $\{G, F\}$ once. This is what the current Phase 2 plan above assumes; it is what makes "community discovery" a non-trivial random variable over Z.

2. *Per-trial frequentist statement about a single fixed recommender*: Z is the operating mode of one recommender in production, G/F the fraction of the time it is well/poorly behaved. Then $P(Z = G)$ is a long-run rate, not a draw.

3. *Subjective uncertainty over a fixed-but-unknown recommender*: Z is the agent's (or modeller's) credence over the recommender's type. Then $P(Z = G)$ is epistemic, not aleatoric.

The reading matters for the conditioning step in the next paragraph: under (1) the per-simulation correlation across agents (the §6 mechanism) follows automatically because Z is shared; under (3) the agent should arguably condition on R_i^t jointly with their evolving posterior *over* Z, not on a sampled Z value. And under (2) it is unclear whether the community-level calculation in §5 (which treats Z as constant across the n agents in a given community) is even well-posed — it would need the recommender's mode to be constant across all n agents within one community, which is a stronger claim than "F happens 1% of the time". Pinning this down before Phase 2 implementation.

In the aided regime, every agent remains a myopic Bayesian but conditions on the AI recommendation before deciding which arm to pull. So, in the aided regime, first an agent conditions on his history to get the posterior π_i^t. Then he conditions on R_i^t. Finally, he chooses the arm with the higher $\pi_i^t(\cdot \mid R_i^t)$ expected payoff.

Pending: `BetaBernoulliAgent.apply_recommendation` currently raises `NotImplementedError`. To implement it we need a likelihood $P(R \mid \text{better arm}, h, \pi, Z)$ on the `Recommendation.likelihood` field. The agent's Bayes update on R then folds that likelihood into the per-arm Beta posterior. *Big modelling choice still open*: does the agent know $P(Z = G)$ and the per-state likelihoods? The paper's setup suggests yes (so the conditioning is a closed-form Bayes update on Z first, then on R), but worth confirming.

4 Model conjectures

I'm guessing that recommenders and recommendation likelihoods can be defined so that, say,

$$P(D_i \mid Z = G) = 0.1, \qquad P(D_i \mid Z = F) = 0.$$

Pending: pick a concrete recommendation rule ρ that realises these numbers. Candidate from §4: in G, R matches the better arm A with probability $1 - \eta_G$ (tunable); in F, $R = B$ with probability $1 - \eta_F$, concentrated at early stages where it bites hardest. Both η_G and η_F are then sweep parameters to calibrate against the targets 0.1 and ≈ 0.

One idea is that in the good state, the recommender is somewhat helpful and often nudges the agent toward the better arm A, raising the probability of discovery from 0.05 to 0.1. In the failure condition, the recommender gives misleading B recommendations that prevent success almost surely. Suppose, for example, that at early stages, when continued sampling of A is important, the recommender often recommends B. If there are enough early B recommendations, the agent won't discover that A is the good arm.

The "early B bias in F" suggests ρ has time-dependent behaviour ($\eta_F = \eta_F(t)$), not just state-dependent. That is a non-trivial complication for the conditioning step in §3: the agent's likelihood must depend on t too. Worth deciding before Phase 2 implementation: do we want stationary or non-stationary ρ?

5 Individuals improve, the community gets worse

For all i, in the aided regime, the probability that i succeeds is

$$(0.99)(0.1) + (0.01)(0) = 0.099.$$

So for each individual the probability of discovery improves from 0.05 to 0.099.

The community calculation is different because Z is common across all agents. Conditional on $Z = G$, the agents still have independent private histories, and each has success probability 0.1. So

the probability that at least one of the 100 agents succeeds is

$$1-(1-0.1)^{100} \approx 0.99997.$$

Conditional on $Z = F$, no one succeeds.

So, the aided community discovery probability is

$$(0.99)\left[1-(1-0.1)^{100}\right] + (0.01)(0) \approx 0.98997.$$

The unaided community success probability is about 0.994. So the AI raises every individual's probability of discovery while lowering the community's probability of discovery.

Phase 2 success criterion: a notebook `2. Phase 2 - Aided Bandit.ipynb` that reports $\hat{q}_{\text{aided}} \approx 0.099$ and $\widehat{P(D)}_{\text{aided}} \approx 0.99$, conditioning explicitly on the recorded Z values per trial. Sweep η_G, η_F to show the headline result is robust, not knife-edge.

6 The mechanism

The mechanism is the introduction of dependence from the common AI recommender. In the unaided regime, some agents get unlucky and abandon A, but others continue sampling it. Their unluckiness is not correlated, so the community benefits from independent variation in payoff histories.

In the aided regime, the agents still have private evidence, but their evidence now includes recommendations generated by the same AI. In the good condition, this common system helps many agents. In the rare failure condition, it moves many posteriors in the same wrong direction. In the aided regime unluckiness becomes correlated. That is the sense in which this AI recommendation can decrease epistemic diversity.

Finally, note that the model is not a wheel-shaped network with AI as the central node. The AI does not sample from the arms or generate any payoff history.

Phase 3 (the empirical verification of the mechanism): quantify the correlation explicitly. For each trial compute (a) the cross-agent correlation of $\hat{p}_A$ trajectories, (b) the conditional correlation given Z, and (c) the residual after removing the Z-mean. Predict: cross-agent correlation is ≈ 0 in the unaided regime, > 0 in the aided regime, and the gap is entirely attributable to Z.

7 Next steps

1. Fill in the details of the model and run simulations to confirm the conjectures and effects in sections 4 and 5.

2. Think about how to make the model more interesting. For example, we could introduce history-

sharing networks together with AI recommendations, and we could model more sophisticated AI recommenders.

Concrete Phase 2 / Phase 3 punch list distilled from the comments above: (1) decide on the discovery definition (§1); (2) parameterise $(p_A, p_B, T, \text{prior})$ so unaided $\hat{q}$ matches the postulated 0.05 (§2); (3) pick the functional form of ρ and whether it is stationary in t (§4); (4) decide what the agent knows about Z and the per-state likelihoods (§3); (5) record Z on `SimulationResult` and report $\widehat{P(D) \mid Z}$ (§3); (6) verify the mechanism via cross-agent correlations (§6). Stretch goal (history-sharing networks) is genuinely Phase 4+: the existing `e_network_inequality` codebase will be the reference there.